CMPT 295

Unit - Machine-Level Programming

Lecture 11 – Assembly language basics:
memory addressing modes,
`leaq` instruction and
arithmetic & logical operations

Last Lecture

- x86-64 assembly language – ***Data***
  - 16 integer registers of 64 bits each: can manipulate either 1, 2, 4 or 8 bytes (memory address of 8 bytes as well) depending on the register name used
  - Floating point registers of 64 bits each: can manipulate either 4 or 8 bytes depending on the register name used
  - No compound data types such as arrays or structures
- x86-64 assembly language – ***Instructions***
  - **`mov*`** instruction family
    - From register to register
    - From memory to register
    - From register to memory
    - *Cannot do memory-memory transfer with a single `mov*` instruction*
  - ~~Memory addressing modes~~

Review from Lecture 10

- **Requirement**: When hand tracing/writing assembly code …

…

… add a comment at the top of your function in your assembly code describing the **parameter-to-register mapping**

```
swap:
    # xp -> %rdi, yp -> %rsi
    movq    (%rdi), %rax    # L1 = *xp
    movq    (%rsi), %rdx    # L2 = *yp
    movq    %rdx, (%rdi)    # *xp = L2
    movq    %rax, (%rsi)    # *yp = L1
    ret
```

Comment each of your assembly language instruction by explaining what it does using corresponding C statement (if you can) or using pseudocode

Review from Lecture 10

Operand Combinations for **mov***

	Operand 1 **Source**	Operand 2 **Dest**		Source, Dest	in C
mov*	*Immediate*	*Register*	`movq $0x4, %rax`		`result = 0x4;`
		Memory	`movq $-147, (%rax)`		`*result = -147;`
	Register	*Register*	`movq %rax, %rdx`		`var1 = result;`
		Memory	`movq %rax, (%rdx)`		`*var1 = result;`
	Memory	*Register*	`movq (%rax), %rdx`		`var1 = *result;`

Examples of some of the memory addressing modes

*Cannot do memory-memory transfer with a single **mov*** instruction*

Why *can't we do memory-memory transfer with a single* ***mov**** *instruction?*

- No x86-64 assembly instructions can take two memory addresses as operands
- If there were such instruction, it would make for very long machine code
  - Bit pattern: <instruction> <mem address>, <mem address>
    length of instruction + 64 bits + 64 bits
  - Considering that memory only has one data bus and one address bus
- This very long machine code would require a **more complex decoder** unit on microprocessor (i.e., a more complex microprocessor *datapath)*
- **Bottom line**: instruction set architects not very keen on creating such instructions
- **Solution**: use registers since registers are very fast and can easily be used for such transfer

- More info here:

https://stackoverflow.com/questions/33794169/why-isnt-movl-from-memory-to-memory-allowed

Today's Menu

- Introduction
  - C program -> assembly code -> machine level code
- Assembly language basics: data, `move` operation
  - Memory addressing modes
- Operation `leaq` and Arithmetic & logical operations
- Conditional Statement – Condition Code + `cmovX`
- Loops
- Function call – Stack
  - Overview of Function Call
  - Memory Layout and Stack - x86-64 instructions and registers
  - Passing control
  - Passing data – Calling Conventions
  - Managing local data
  - Recursion
- Array
- Buffer Overflow
- Floating-point data & operations

Three types of operands to x86-64 instructions

1. **Immediate** - Constant integer value used as operand in an instruction
   - Operand syntax: `Imm`
   - Examples: `movq $0x4,%rax` and `movb $-17,%al`
2. **Register** used as operand in instruction
   - Operand value: $R[r_a]$
   - Operand syntax: $\%r_a$ ← name of particular register
   - Example: `movq %rax,%rdx`
3. **Memory addressing mode** (expression) used as operand in an instruction

So far, this is the type of operands what we have seen!

These instructions copy **immediate** value to register

This instruction copies the value of one register into another register

Memory addressing modes

We access memory in an x86-64 instruction by expressing a memory address through various ***memory addressing modes***

1. **Absolute** memory addressing mode
   - Use ***memory address*** as operand directly in instruction
     - The operand is also called **immediate**
   - Operand syntax: `Imm`
   - Effect: `M[Imm]`
   - Example: `call plus`

`plus` refers to the memory address of the first byte of the first instruction of the function called `plus` (see `sum_store.s` – Lecture 9 Demo)

2. **Indirect** memory addressing mode

- When a register contains a memory address
  - Similar to a pointer in C
- To access the data at the memory address contained in the register, we use parentheses **(…)**
- General Syntax: **(r_b)**
- Effect: **M[R[r_b]]**
- Example:

```
swap:
   # xp -> %rdi, yp -> %rsi
   movq     (%rdi), %rax   # L1 = *xp
   movq     (%rsi), %rdx   # L2 = *yp
   movq     %rdx, (%rdi)   # *xp = L2
   movq     %rax, (%rsi)   # *yp = L1
   ret
```

2. **Indirect** memory addressing mode

- Example: register to register `movq %rdx,%rax` vs memory to register `movq (%rdx),%rax`

Meaning or effect: `%rdx -> %rax` vs `M[%rdx] -> %rax`

or: `R[%rdx] -> R[%rax]` `M[R[%rdx]] -> R[%rax]`

Before	After	Before	After
%rax = 15	**%rax = 6**	%rax = 15	**%rax = 11**
%rdx = 6	%rdx = 6	%rdx = 6	%rdx = 6
not used M[6] = 11	M[6] = 11	M[6] = 11	M[6] = 11

- Other examples: `movq %rax,(%rdx)` <- register to memory

`movq $-147,(%rax)` <- immediate to memory

leaq - *Load effective address* instruction

Warning!
leaq has the form of an instruction that reads from memory to a register because of the parentheses, however it ***does not*** **reference memory at all!**

- Often used for ***address computations*** and ***general arithmetic computations***
- **Syntax:** `leaq Source, Destination`
- Examples:

1. Computing a memory address: $r_b + r_i$

```
leaq (%rax, %rcx), %rdx
      rb    ri
```

Example:
if %rax contains 0x0000000000000008
and %rcx contains 16,
once executed,
%rdx contains 0x18

2. Computing arithmetic expressions of the form $r_b + r_i * s$ where $s \in \{1,2,4,8\}$

```
leaq (%rdi, %rdi, 2), %rax
      rb    ri   s
```

Example:
if %rdi contains variable a,
once executed,
%rax contains 3a

In C, this would be:
```
return 3*a;
```

- Operand ***Destination*** is a register
- Operand ***Source*** is a ***memory addressing mode expression***

3. "**Base + displacement**" memory addressing mode

displacement base

- General Syntax: *Imm*(r$_b$)
- Effect: M[*Imm* + R[r$_b$]]
- Examples: `movq %rax, -8(%rsp)`

  `leaq 7(%rdi), %rax`
- **Warning! Careful here!**
  - When dealing with **leaq**, the effect is *Imm* + R[r$_b$]

    not M[*Imm* + R[r$_b$]]

4. **Indexed** memory addressing mode

1. General Syntax: (r_b, r_i) (base, index)

- Effect: `M[R[r_b] + R[r_i]]`
- Example: `movb (%rdi, %rcx), %al`

2. General Syntax: $Imm(r_b, r_i)$ (displacement, base, index)

- Effect: `M[Imm + R[r_b] + R[r_i]]`
- Example: `movw 0xA(%rdi, %rcx), %r11w`

Warning! Careful here!

- When dealing with **`leaq`**, the effect is
  1. `R[r_b] + R[r_i]` ***not*** `M[R[r_b] + R[r_i]]`
  2. `Imm + R[r_b] + R[r_i]` ***not*** `M[Imm + R[r_b] + R[r_i]]`

5. **Scaled** indexed memory addressing mode

scale

1. General Syntax: `(,r_i,s)` Effect: `M[R[r_i] * s]`
   - Example: `(, %rdi, 2)`
2. General Syntax: `Imm(,r_i,s)` Effect: `M[Imm + R[r_i] * s]`
   - Example: `3(, %rcx, 8)`
3. General Syntax: `(r_b,r_i,s)` Effect: `M[R[r_b] + R[r_i] * s]`
   - Example: `(%rdi, %rsi, 4)`
4. General Syntax: `Imm(r_b,r_i,s)` Effect: `M[Imm + R[r_b] + R[r_i] * s]`
   - Example: `8(%rdi, %rsi, 4)`

Warning! Again, careful here!

- When dealing with `leaq`, the effect is ***not*** to reference **memory at all!**

Summary - Memory addressing modes

In an x86-64 instruction, we access memory (via the operand(s)) by expressing a memory address using various ***memory addressing modes***

1. **Absolute**
2. **Indirect**
3. **"Base + displacement"**
4. **2 indexed**
5. **4 scaled indexed**

General Syntax: `Imm(r_b, r_i, s)`

Effect: `M[Imm + R[r_b]+ R[r_i] * s]`

See Table of x86-64 Addressing Modes on Resources web page of our course web site

From: https://cs61.seas.harvard.edu/site/2019/Asm/#Buffer-overflow-attacks

Address modes

Most instruction operands use the following syntax for values. (See also CS:APP3e Figure 3.3 in §3.4.1, p181.)

Type	Example syntax	Value used
Register	`%rbp`	Contents of `%rbp`
Immediate	`$0x4`	0x4
Memory	`0x4`	Value stored at address 0x4
	`symbol_name`	Value stored in global `symbol_name` (the compiler resolves the symbol name to an address when creating the executable)
	`symbol_name(%rip)`	`%rip`-relative addressing for global
	`symbol_name+4(%rip)`	Simple computations on symbols are allowed (the compiler resolves the computation when creating the executable)
	`(%rax)`	Value stored at address in `%rax`
	`0x4(%rax)`	Value stored at address `%rax + 4`
	`(%rax,%rbx)`	Value stored at address `%rax + %rbx`
	`(%rax,%rbx,4)`	Value stored at address `%rax + %rbx*4`
	`0x18(%rax,%rbx,4)`	Value stored at address `%rax + 0x18 + %rbx*4`

The full form of a memory operand is `offset(base,index,scale)`, which refers to the address `offset + base + index*scale`. In `0x18(%rax,%rbx,4)`, `%rax` is the base, `0x18` the offset, `%rbx` the index, and `4` the scale. The offset (if used) must be a constant and the base and index (if used) must be registers; the scale must be either 1, 2, 4, or 8. The default offset, base, and index are 0, and the default scale is 1.

From: https://cs61.seas.harvard.edu/site/2019/Asm/#Buffer-overflow-attacks

Address computations

The `offset(base,index,scale)` form compactly expresses many array-style address computations. It's typically used with a `mov`-type instruction to dereference memory. However, the compiler can use that form to compute addresses, thanks to the `lea` (Load Effective Address) instruction.

For instance, in `movl 0x18(%rax,%rbx,4), %ecx`, the address `%rax + 0x18 + %rbx*4` is computed, then immediately dereferenced: the 4-byte value located there is loaded into `%ecx`. In `leaq 0x18(%rax,%rbx,4), %rcx`, the same address is computed, but it is *not* dereferenced. Instead, the *computed address* is moved into register `%rcx`.

Thanks to `lea`, the compiler will also prefer the `offset(base,index,scale)` form over `add` and `mov` for certain computations on integers. For example, this instruction:

```
leaq (%rax,%rbx,2), %rcx
```

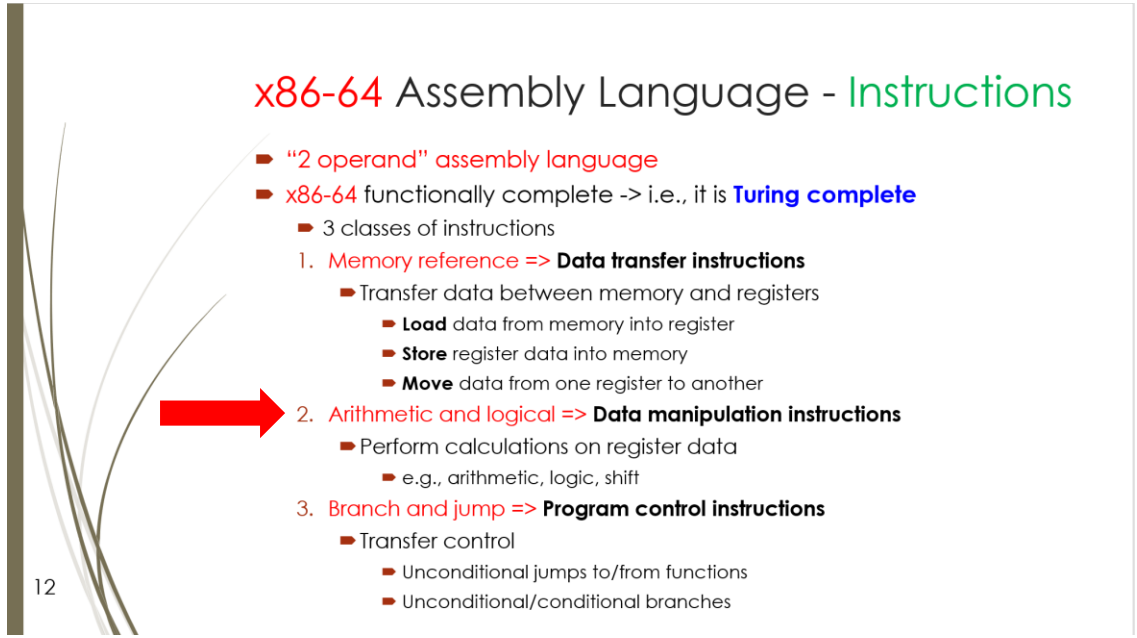
performs the function `%rcx := %rax + 2 * %rbx`, but in one instruction, rather than three (`movq %rax, %rcx; addq %rbx, %rcx; addq %rbx, %rcx`).

Let's try it!

%rdx	0x000000000000f000
%rcx	0x0000000000000100

Expression	Address Computation	Address
1. 8(%rdx)		
2. (%rdx,%rcx)		
3. (%rdx,%rcx,4)		
4. 0x80(,%rdx,2)		
5. 0x80(%rdx, 2)		
6. 0x80(,%rdx, 3)		

Next class of instructions



- Remember Slide 12 from our Lecture 10:
  and the fact that x86-64 is a functionally complete assembly language
  i.e., that it is **Turing complete**
- We have covered the first class of instructions:
  - Memory reference => **Data transfer instructions**
- Now, let's move on to
  - Arithmetic and logical => **Data manipulation instructions**

```
* -> Size designator
 q -> long  64
 l -> int   32
 w -> short 16
 b -> char   8
```

Two-Operand Arithmetic Instructions

Syntax	Meaning	Examples	in C
add* ***Src, Dest***	**Dest ← Dest + Src**	`addq %rax, %rcx`	`x += y`
sub* ***Src, Dest***	**Dest ← Dest – Src**	`subq %rax, %rcx`	`x -= y`
imul* ***Src, Dest***	**Dest ← Dest * Src**	`imulq $16,(%rax,%rdx,8)`	`x *= y`

- **mem ← mem OP mem** usually not supported
- 2 assembly code formats: **ATT and Intel format** (see Aside in Section 3.2 P. 177)
  - We are using the **ATT format** (as opposed to the **Intel** format)
  - Both order the operands of their instructions differently - Watch out!

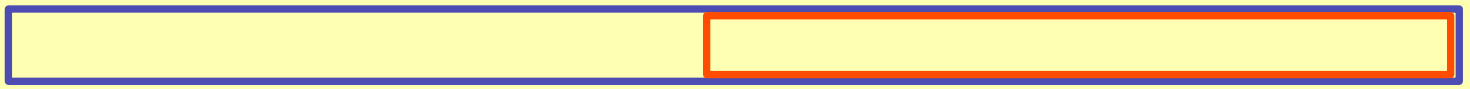
* -> Size designator

```
q -> long  64
l -> int   32
w -> short 16
b -> char   8
```

Two-Operand Logical Instructions

Syntax	Meaning	Examples
`and* Src, Dest`	`Dest ← Dest & Src`	`andl $252645135, %edi`
`or* Src, Dest`	`Dest ← Dest | Src`	`orq %rsi, %rdi`
`xor* Src, Dest`	`Dest ← Dest ^ Src`	`xorq %rsi, %rdi`

- **`xorq`** special purpose:
  - **`xorq %rax, %rax`** <- *zeroes* register `%rax`
  - **`movq $0, %rax`** <- also *zeroes* register `%rax`
- x86-64 convention:
  - Any instruction updating the 4 LSB will cause the 4 MSB bytes to be set to 0
  - **`xorl %eax, %eax`** and **`movl $0, %eax`** <- also *zeroes* register `%rax`



```
* -> Size designator
 q -> long   64
 l -> int    32
 w -> short  16
 b -> char    8
```

Two-Operand Shift Instructions

Syntax	Meaning	Examples
`sal*` ***`Src, Dest`***	**`Dest ← Dest << Src`**	**`salq $4, %rax`**

- *Left shift* - also called **`shlq`**: filling **`Dest`** with 0, from the right

`sar*` ***`Src, Dest`***	**`Dest ← Dest >> Src`**	**`sarl %cl, %rax`**

- *Right arithmetic Shift*: filling **`Dest`** with sign bit, from the left

`shr*` ***`Src, Dest`***	**`Dest ← Dest >> Src`**	**`shrq $2, %r8`**

- *Right logical Shift*: filling **`Dest`** with 0, from the left

* -> Size designator

```
q -> long  64
l -> int   32
w -> short 16
b -> char   8
```

One-Operand Arithmetic Instructions

Syntax	Meaning	Examples
`inc*` *Dest*	Dest ← Dest + 1	`incq (%rsp)`
`dec*` *Dest*	Dest ← Dest – 1	`decb %dil`
`neg*` *Dest*	Dest ← –Dest	`negl %eax`
`not*` *Dest*	Dest ← ~Dest	`notq %rdi`

Summary

- x86-64 instructions can have various types of **operands**
  1. Immediate (constant integral value)
  2. Register (16 registers)
  3. Memory addressing mode (various forms)
     - General Syntax of operand: **Imm(r_b, r_i, s)**
- **leaq** - load effective address instruction
- Arithmetic & logical instructions
  - Arithmetic instructions: **add***, **sub***, **imul*** **inc***, **dec***, **neg***, **not***
  - Logical instructions: **and***, **or***, **xor***
  - Shift instructions: **sal***, **sar***, **shr***

* -> Size designator

```
q -> long  64
l -> int   32
w -> short 16
b -> char   8
```

Next lecture

- Introduction
  - C program -> assembly code -> machine level code
- Assembly language basics: data, `move` operation
  - Memory addressing modes
- Operation `leaq` and Arithmetic & logical operations
- Conditional Statement – Condition Code + `cmovX`
- Loops
- Function call – Stack
  - Overview of Function Call
  - Memory Layout and Stack - x86-64 instructions and registers
  - Passing control
  - Passing data – Calling Conventions
  - Managing local data
  - Recursion
- Array
- Buffer Overflow
- Floating-point data & operations

Practice and DEMO!